

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplexer	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A \mid B$, $A \& B$); multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawi $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Zbudowany z dwóch sprzężonych NOR -ów.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R bramkowane AND-em z zegarem. Stan zmienia się tylko przy CLK=1 .
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Bity, bajty i typy danych

1B = 8b, hex = 4b $\Rightarrow$ 1B $\in$ [0x00; 0xFF]. Np. 15213 = 0x3B6D.

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$
$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2). Stąd **B2U** = bity $\rightarrow$ unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (INT_MAX)
TMin	10...0	-2^{w-1} (INT_MIN)
−1	11...1	te same bity co UMax

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ($< > ==$) $\rightarrow$ **int** promowany do **unsigned** (bity bez zmian, $-1 \rightarrow$ UMax).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (zext)	unsigned : dopisuje 0 z góry
Rozszerz. znak (sext)	signed : powiela bit znaku (MSB)
Obcięcie (trunc)	$u \bmod 2^w$; może zmienić znak
$x \ll k$	$x \cdot 2^k$ (sgn./uns.)
$u \gg k$	$\lfloor u/2^k \rfloor$ — logiczne (zera)
$x \gg k$	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	$(x + (1 \ll k) - 1) \gg k$

Strength red.: $x * 24 \rightarrow (x \ll 5) - (x \ll 3)$.

5. Operacje, overflow i endianness

UAdd	$(u + v) \bmod 2^w$; carry ignorowane
TAdd	bitowo = UAdd; różne znaki nie przepełniają
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)
Nadmiar (-)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)
Negacja U2	$-x = \sim x + 1$; $-\text{TMin} = \text{TMin}$, $-0 = 0$
Wykr. nadmiaru (s=x+y)	$((s \wedge x) \& (s \wedge y)) \gg (w-1)$
Maska / abs	$m = x \gg 31$; $\text{abs} = (x \wedge m) - m$

Pamięć: adres wskazuje najmłodszy bajt słowa. Napis = **char[]** ASCII + **0x00** (niezależny od endianness).

Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][123]
Little Endian	LSB	[23][01]

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

Format	bity	s	exp (k)	frac (n)
Single (float)	32	1	8 (Bias=127)	23
Double (double)	64	1	11 (Bias=1023)	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	!= 0...0 != 1...1	$E = \text{exp} - \text{Bias}$. Ukryta jedyńska: $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	== 0...0	$E = 1 - \text{Bias}$. Brak jedyнки: $M = 0.\text{frac}$. Reprezentują +0.0 i -0.0 (frac = 0) oraz liczby bardzo bliskie zera (stopniowy underflow).
Specjalne	== 1...1	Gdy frac == 0, to $+\infty$ lub $-\infty$ (nadmiar/dzielenie przez 0). Gdy frac $\neq$ 0, to NaN (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty

Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	= 0.5	Do parzystej: w górę gdy G=1, w dół gdy G=0

Własności matematyczne i rzutowanie

- Brak łączności: $(a + b) + c \neq a + (b + c)$, przez zaokrąglenia i utratę danych.
- Brak rozdzielności: $a(b + c) \neq ab + ac$.
- Rzutowanie int → float: Może stracić precyzję (zaokrągła zgodnie z trybem).
- Rzutowanie int → double: Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- Rzutowanie float/double → int: Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca Tmin (0x80000000).

7. Instrukcja leaq (Load Effective Address)

Instrukcja obliczeniowa, nie pamięciowa. Ładuje obliczony adres, a nie wartość z pamięci: leaq Src, Dst → Dst = addr(Src).

Zastosowania:

- Pobieranie adresu zmiennej: C-owe p = &x[i].
- Szybka arytmetyka bez flag: Obliczenia $x + k \cdot y$ dla stałych $k \in \{1, 2, 4, 8\}$.

Przykład: Optymalizacja x * 12:

```
leaq (%rdi, %rdi, 2), %rax ; t = x + 2*x = 3*x
salq $2, %rax ; return t << 2 (czyli 3*x * 4 = 12*x)
```

8. Rejestry x86_64

%rax	%eax %ah %al	%rbx	%ebx %bh %bl
%rcx	%ecx %ch %cl	%rdx	%edx %dh %dl
%rsi	%esi %sil	%rdi	%edi %dil
%rbp	%ebp %bp %bpl	%rsp	%esp %spl
%r8	%r8d %r8w %r8b	%r9	%r9d %r9w %r9b

Uwaga: Rejestry od %r10 do %r15 używają schematu: %r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

%rax	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną.
%rdi, %rsi, %rdx, %rcx, %r8, %r9	Kolejno: od 1. do 6. argumentu funkcji. Caller-saved.
%rsp	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
%rbp	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
%rbx, %r12-%r15	Ogólnego przeznaczenia. Wszystkie callee-saved.
%rip	Wskaźnik instrukcji (Program Counter).

9. Tryby adresowania (operands)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	\$Imm	Imm
Rejestrowy (Reg)	%Ra	%Ra
Bezpośredni (Mem)	Imm	M[Imm]
Pośredni (Mem)	C(%Rb)	M[%Rb]
Z przesunięciem	D(%Rb)	M[%Rb + DI]
Skalowany (Ind/scaled)	D(%Rb, %Ri, S)	M[%Rb + %Ri * S + DI]
Skalowany (baseless)	(, %Ri, S)	M[%Ri * S]

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

10. Podstawowe instrukcje i arytmetyka (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sar / shl k, D</code>	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
<code>shr k, D</code>	$D \gg = k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często używane jako <code>xor %rax, %rax</code> do zerowania.

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).